

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Информационная безопасность баз данных»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2

«Реализация БД в рамках СУБД»

Выполнили:

Матвеев Илья Александрович, студент группы N3351

(подпись)

Проверил:

Карманова Наталия Андреевна, ассистент ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

Содержание	2
Введение	3
1 Ход Работы	4
1.1 Выбор СУБД и обоснование выбора	4
1.2 Создание базы данных и создание таблиц	4
1.3 Создание триггеров	5
1.4 Наполнение таблиц данными	9
1.5 Индексирование и связь таблиц	13
1.6 Создание представлений	14
Заключение	16

ВВЕДЕНИЕ

Цель работы – получение навыков по работе с современными системами управления базами данных.

Для достижения поставленной цели необходимо решить следующие задачи:

– Выбрать систему управления базами данных, которая будет использоваться при выполнении работы, обосновать выбор;

- Создать БД, разработанную в лабораторной работе №1;
- Наполнить БД кортежами данных;
- Индексировать таблицы;
- Установить взаимосвязи между таблицами;
- Создать представления, описанные в лабораторной работе №1.

1 ХОД РАБОТЫ

Данный раздел посвящен выполнению заданий, описанных во введении.

1.1 Выбор СУБД и обоснование выбора

Выбранная мной система управления базами данных – PostgreSQL и визуальный интерфейс pgAdmin 4. Ссылка для ознакомления с данной системой - <https://www.postgresql.org/>. Мой выбор основывается на том, что в прошлых проектах я использовал именно эту СУБД, соответственно имею обширный опыт работы с ней.

1.2 Создание базы данных и создание таблиц

Основное уточнение – база данных будет создана на локальном сервере в качестве упражнения для лабораторных работ. В будущем планируется открыть собственный сервер, что будет связано с затратами для его поддержания.

После создания онлайн базы Online-school приступим к созданию таблиц, описанных в лабораторной работе №1. Код создания таблиц представлен в листинге 1.

Листинг 1 – Код создания таблиц.

```
CREATE TABLE Students (  
    StudentID SERIAL PRIMARY KEY,  
    FIO VARCHAR(100) NOT NULL,  
    DateOfBirth DATE,  
    Email VARCHAR(100) UNIQUE,  
    Phone VARCHAR(100) UNIQUE,  
    RegistrationDate DATE  
);  
CREATE TABLE Teachers (  
    TeacherID SERIAL PRIMARY KEY,  
    FIO VARCHAR(100) NOT NULL,  
    Specialization VARCHAR(100) NOT NULL,  
    Experience INT,  
    Email VARCHAR(100) UNIQUE  
);  
CREATE TABLE Courses (  
    CourseID SERIAL PRIMARY KEY,  
    Title VARCHAR(100) NOT NULL,  
    Description TEXT,  
    Price NUMERIC(10,2),  
    Duration INTERVAL,  
    TeacherID INT REFERENCES Teachers(TeacherID) ON DELETE SET NULL  
);  
CREATE TABLE Groups (  
    GroupID SERIAL PRIMARY KEY,  
    Title VARCHAR(100) NOT NULL,  
    CourseID INT REFERENCES Courses(CourseID) ON DELETE CASCADE,  
    CreationDate DATE  
);  
CREATE TABLE StudentGroups (  
    StudentID INT REFERENCES Students(StudentID) ON DELETE SET NULL,  
    GroupID INT REFERENCES Groups(GroupID) ON DELETE SET NULL,  
    DateOfRegistration DATE
```

```

        StudentID INT REFERENCES Students(StudentID) ON DELETE CASCADE,
        GroupID INT REFERENCES Groups(GroupID) ON DELETE CASCADE,
        PRIMARY KEY (StudentID, GroupID)
    );
CREATE TYPE assignment_type AS ENUM ('Домашнее задание', 'Контрольная
работа', 'Тест');
CREATE TABLE Assignments (
    AssignmentID SERIAL PRIMARY KEY,
    CourseID INT REFERENCES Courses(CourseID) ON DELETE CASCADE,
    Title VARCHAR(100) NOT NULL,
    As_Type assignment_type,
    Deadline DATE,
    Description TEXT
);
CREATE TABLE Results (
    ResultID SERIAL PRIMARY KEY,
    AssignmentID INT REFERENCES Assignments(AssignmentID) ON DELETE CASCADE,
    StudentID INT REFERENCES Students(StudentID) ON DELETE CASCADE,
    Grade INT CHECK (Grade BETWEEN 0 AND 100),
    SubmissionDate DATE,
    Comment_fr TEXT
);
CREATE TABLE Mentors (
    MentorID SERIAL PRIMARY KEY,
    FIO VARCHAR(100) NOT NULL,
    SubjectArea VARCHAR(100),
    Email VARCHAR(100) UNIQUE,
    AppointmentDate DATE
);
CREATE TABLE MentorResults (
    MentorID INT REFERENCES Mentors(MentorID) ON DELETE CASCADE,
    ResultID INT REFERENCES Results(ResultID) ON DELETE CASCADE,
    PRIMARY KEY (MentorID, ResultID)
);
CREATE TABLE Schedule (
    ScheduleID SERIAL PRIMARY KEY,
    GroupID INT REFERENCES Groups(GroupID) ON DELETE CASCADE,
    TeacherID INT REFERENCES Teachers(TeacherID) ON DELETE SET NULL,
    CourseID INT REFERENCES Courses(CourseID) ON DELETE CASCADE,
    Date DATE,
    Time TIME,
    LessonType VARCHAR(50)
);
CREATE TABLE Logs (
    LogID SERIAL PRIMARY KEY,
    UserID INT NOT NULL,
    UserRole VARCHAR(50) CHECK (UserRole IN
('Student', 'Teacher', 'Mentor', 'Admin')),
    Action TEXT NOT NULL,
    ActionTime TIMESTAMP DEFAULT now()
);

```

1.3 Создание триггеров

Перед наполнением БД данными необходимо позаботиться о логике работы, и создать триггеры, которые будут отвечать за аудит, автоматизацию и валидность добавляемых данных.

Приведу описание минимально необходимых для работы системы триггеров:

- Автозаполнение атрибута «дата_регистрации» ученика – позволяет системе устанавливать время регистрации как текущее, если не указано иное;
- Логирование регистрации для учеников, преподавателей и менторов – для дальнейшего аудита каждые данные будут логироваться и добавляться в таблицу с логами;
- Логирование добавления студента в группу – заносит в логи запись о том, что ученик добавлен в группу;
- Логирование сдачи или обновления результата ученика – позволяет отслеживать получение результатов;
- Проверка валидности дат – проверяет даты рождения, даты дедлайнов, даты проверок заданий.

В листинге 2 представлен код, реализующий каждый из этих триггеров.

Листинг 2 – Создание триггеров.

```
CREATE OR REPLACE FUNCTION set_registration_date()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.RegistrationDate IS NULL THEN
        NEW.RegistrationDate := CURRENT_DATE;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_set_registration_date
BEFORE INSERT ON Students
FOR EACH ROW
EXECUTE FUNCTION set_registration_date();

CREATE OR REPLACE FUNCTION log_student_registration()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO Logs(UserID, UserRole, Action)
    VALUES (NEW.StudentID, 'Student', 'Регистрация нового ученика');
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_log_student_registration
AFTER INSERT ON Students
FOR EACH ROW
EXECUTE FUNCTION log_student_registration();

CREATE OR REPLACE FUNCTION log_teacher_registration()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO Logs(UserID, UserRole, Action)
    VALUES (NEW.TeacherID, 'Teacher', 'Регистрация нового преподавателя');
    RETURN NEW;
END;
```

```

$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_log_teacher_registration
AFTER INSERT ON Teachers
FOR EACH ROW
EXECUTE FUNCTION log_teacher_registration();

CREATE OR REPLACE FUNCTION log_mentor_registration()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO Logs(UserID, UserRole, Action)
    VALUES (NEW.MentorID, 'Mentor', 'Регистрация нового ментора');
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_log_mentor_registration
AFTER INSERT ON Mentors
FOR EACH ROW
EXECUTE FUNCTION log_mentor_registration();

CREATE OR REPLACE FUNCTION log_student_added_to_group()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO Logs(UserID, UserRole, Action)
    VALUES (NEW.StudentID, 'Student',
            'Добавлен в группу ID=' || NEW.GroupID);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_log_student_added_to_group
AFTER INSERT ON StudentGroups
FOR EACH ROW
EXECUTE FUNCTION log_student_added_to_group();
CREATE OR REPLACE FUNCTION log_result_submission()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO Logs(UserID, UserRole, Action)
    VALUES (NEW.StudentID, 'Student',
            'Сдано задание ID=' || NEW.AssignmentID ||
            ', оценка: ' || COALESCE(NEW.Grade::TEXT, 'без оценки'));
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_log_result_submission
AFTER INSERT ON Results
FOR EACH ROW
EXECUTE FUNCTION log_result_submission();

CREATE OR REPLACE FUNCTION log_result_update()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.Grade IS DISTINCT FROM OLD.Grade THEN
        INSERT INTO Logs(UserID, UserRole, Action)
        VALUES (NEW.StudentID, 'Student',
                'Изменена оценка по заданию ID=' || NEW.AssignmentID ||
                ': было ' || OLD.Grade || ', стало ' || NEW.Grade);
    END IF;
    RETURN NEW;
END;

```

```

$$ LANGUAGE plpgsql;

CREATE OR REPLACE FUNCTION validate_dates()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_TABLE_NAME = 'students' THEN
        IF NEW.DateOfBirth > CURRENT_DATE THEN
            RAISE EXCEPTION 'Дата рождения не может быть в будущем';
        END IF;
    END IF;
    IF TG_TABLE_NAME = 'assignments' THEN
        IF NEW.Deadline < CURRENT_DATE THEN
            RAISE EXCEPTION 'Дедлайн не может быть в прошлом';
        END IF;
    END IF;
    IF TG_TABLE_NAME = 'results' THEN
        IF NEW.SubmissionDate > (SELECT Deadline
                                FROM Assignments
                                WHERE AssignmentID = NEW.AssignmentID) THEN
            RAISE EXCEPTION 'Дата сдачи не может быть позже дедлайна
задания';
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validate_student_dates
BEFORE INSERT OR UPDATE ON Students
FOR EACH ROW
EXECUTE FUNCTION validate_dates();

CREATE TRIGGER trg_validate_assignment_dates
BEFORE INSERT OR UPDATE ON Assignments
FOR EACH ROW
EXECUTE FUNCTION validate_dates();

CREATE TRIGGER trg_validate_result_dates
BEFORE INSERT OR UPDATE ON Results
FOR EACH ROW
EXECUTE FUNCTION validate_dates();

CREATE OR REPLACE FUNCTION set_group_creation_date()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.creationdate IS NULL THEN
        NEW.creationdate := CURRENT_DATE;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_set_group_creation_date
BEFORE INSERT ON Groups
FOR EACH ROW
EXECUTE FUNCTION set_group_creation_date();

```

1.4 Наполнение таблиц данными

Для демонстрации работы бизнес-логики предстоящего приложения необходимо заполнить таблицы кортежами данных. Представим, что в системе зарегистрируются 13 учеников, в системе будет 6 курсов, 7 преподавателей и 10 менторов, а также в каждом курсе будет по 3 задания (18 записей), и каждый ученик сделает хотя бы 1 задание из курса (13 записей в таблице результаты). Кроме того, необходимо каждому ученику сделать расписание, уточнить расписание для преподавателей, что оценивается в 20 записей. Код создания кортежей данных представлен ниже.

Листинг 3 – Добавление кортежей данных

```
#Ученики
INSERT INTO Students (FIO, DateOfBirth, Email, Phone)
VALUES ('Иванов Иван Иванович', '2007-05-14', 'ivanov@example.com',
'+79998887766');
INSERT INTO Students (FIO, DateOfBirth, Email, Phone)
VALUES ('Золин Алексей Игоревич', '2007-09-12', 'zolinai@mail.ru',
'+79601243275');
INSERT INTO Students (FIO, DateOfBirth, Email, Phone)
VALUES ('Кирин Никита Владимирович', '2007-03-25', 'kirinnv@gmail.com',
'+79205167283');
INSERT INTO Students (FIO, DateOfBirth, Email, Phone)
VALUES ('Васильев Пётр Петрович', '2007-11-09', 'пётр.васильев72@yandex.ru',
'+79410761072');
INSERT INTO Students (FIO, DateOfBirth, Email, Phone)
VALUES ('Смирнов Пётр Петрович', '2007-11-11', 'пётр.смирнов65@example.com',
'+79613209970');
INSERT INTO Students (FIO, DateOfBirth, Email, Phone)
VALUES ('Сидоров Иван Иванович', '2007-06-22', 'иван.сидоров47@example.com',
'+79096106152');
INSERT INTO Students (FIO, DateOfBirth, Email, Phone)
VALUES ('Васильев Пётр Петрович', '2007-11-11', 'пётр.васильев50@gmail.com',
'+79037031799');
INSERT INTO Students (FIO, DateOfBirth, Email, Phone)
VALUES ('Иванов Пётр Иванович', '2007-02-25', 'пётр.иванов86@yandex.ru',
'+79899867866');
INSERT INTO Students (FIO, DateOfBirth, Email, Phone)
VALUES ('Сидоров Пётр Алексеевич', '2007-03-14', 'пётр.сидоров80@mail.ru',
'+79800265436');
INSERT INTO Students (FIO, DateOfBirth, Email, Phone)
VALUES ('Иванов Никита Алексеевич', '2007-11-12',
'никита.иванов52@example.com', '+79633438111');
INSERT INTO Students (FIO, DateOfBirth, Email, Phone)
VALUES ('Кузнецов Андрей Андреевич', '2007-03-11',
'андрей.кузнецов84@yandex.ru', '+79583628333');
INSERT INTO Students (FIO, DateOfBirth, Email, Phone)
VALUES ('Кузнецов Дмитрий Никитич', '2007-06-28',
'дмитрий.кузнецов77@gmail.com', '+79232988515');
INSERT INTO Students (FIO, DateOfBirth, Email, Phone)
VALUES ('Петров Андрей Дмитриевич', '2007-04-23',
'андрей.петров80@example.com', '+79991428850');
#Преподаватели
INSERT INTO Teachers (FIO, specialization, experience, email)
VALUES ('Мельников Дмитрий Евгеньевич', 'Программирование', '10',
'melnikovde@gmail.com');
```

```

INSERT INTO Teachers(FIO, specialization, experience, email)
VALUES ('Штормовая Полина Андреевна', 'Программирование', '2',
'polinashtorm@gmail.com');
INSERT INTO Teachers(FIO, specialization, experience, email)
VALUES ('Некрылов Евгений Фёдорович', 'Школьная математика', '3',
'coolmathematic@gmail.com');
INSERT INTO Teachers(FIO, specialization, experience, email)
VALUES ('Александровская Наталья Николаевна', 'Высшая математика', '20',
'alexandrovskayani@gmail.com');
INSERT INTO Teachers(FIO, specialization, experience, email)
VALUES ('Никольский Николай Павлович', 'Информатика и ИКТ', '5',
'velnik@gmail.com');
INSERT INTO Teachers(FIO, specialization, experience, email)
VALUES ('Джон Скоузер', 'Английский язык', '15', 'englishteacher@gmail.com');
INSERT INTO Teachers(FIO, specialization, experience, email)
VALUES ('Казаринова Людмила Павловна', 'Школьная физика', '2',
'ludmilaphys@gmail.com');
#Курсы
INSERT INTO Courses(title, description, price, duration, teacherid)
VALUES ('Программирование', 'Курс позволит участникам изучить
программирование на популярных языках с нуля. Начальных знаний не требуется.
Длительность
курса - 6 месяцев. Курс сочетает большое количество практики и теории. Курс
ведут опытные педагоги, работающие в крупных IT компаниях.', '25000',
'months', '1');
INSERT INTO Courses(title, description, price, duration, teacherid)
VALUES ('Программирование', 'Курс позволит участникам изучить
программирование на популярных языках с нуля. Начальных знаний не требуется.
Длительность
курса - 6 месяцев. Курс сочетает большое количество практики и теории. Курс
ведут опытные педагоги, работающие в крупных IT компаниях.', '25000', '6
months', '2');
INSERT INTO Courses(title, description, price, duration, teacherid)
VALUES ('Школьная математика', 'Курс позволит школьникам с 9го до 11го класса
лучше разобраться в том, как работает математика. Прохождение курса позволит
лучше выполнять домашние задания, сдавать контрольные, и готовиться к ЕГЭ.
Длительность курса - 12 месяцев', '45000', '12 months', '3');
INSERT INTO Courses(title, description, price, duration, teacherid)
VALUES ('Высшая математика', 'Курс высшей математики рассчитан на студентов
1го курса технического университета. В рамках курса участники пройдут важные
сферы вузовской математики - от интегралов до теории поля.', '15000', '3
months', '4');
INSERT INTO Courses(title, description, price, duration, teacherid)
VALUES ('Информатика и ИКТ', 'Курс поможет школьникам освоить базу в
программированию, научит архитектуре компьютера, основам компьютерных сетей,
позволит лучше подготовиться к ЕГЭ по информатике. Курс рассчитан на учеников
10-11 классов', '20000', '3 months', '5');
INSERT INTO Courses(title, description, price, duration, teacherid)
VALUES ('Английский язык', 'Курс поможет разобраться в профессиональном
английском языке для программистов. Курс ведет опытный носитель языка',
'95000',
'12 months', '6');
INSERT INTO Courses(title, description, price, duration, teacherid)
VALUES ('Школьная физика', 'Курс поможет школьникам освоить базу физики - от
механики до квантовой физики. Курс поможет подготовиться к ЕГЭ
по физике', '15000', '3 months', '7');
#Группы
INSERT INTO Groups(title, courseid)
VALUES ('1 группа программирование', '1');
INSERT INTO Groups(title, courseid)
VALUES ('2 группа программирование', '4');
INSERT INTO Groups(title, courseid)

```

```

VALUES ('1 группа ИМ', '2');
INSERT INTO Groups(title, courseid)
VALUES ('1 группа ВМ', '3');
INSERT INTO Groups(title, courseid)
VALUES ('1 группа ИКТ', '5');
INSERT INTO Groups(title, courseid)
VALUES ('1 группа Английский', '6');
INSERT INTO Groups(title, courseid)
VALUES ('1 группа ШФ', '7');
#Задания
INSERT INTO Assignments (courseid, title, as_type, deadline, description)
VALUES ('1', 'Первое задание', 'Домашнее задание', '2025-09-28', 'Начальные
навыки');
INSERT INTO Assignments (courseid, title, as_type, deadline, description)
VALUES ('1', 'Второе задание', 'Домашнее задание', '2025-10-12', 'Начальные
навыки - отработка');
INSERT INTO Assignments (courseid, title, as_type, deadline, description)
VALUES ('1', 'Проверка', 'Контрольная работа', '2025-10-28', 'Проверка
знаний');
INSERT INTO Assignments (courseid, title, as_type, deadline, description)
VALUES ('2', 'Первое задание', 'Домашнее задание', '2025-09-28', 'Начальные
навыки');
INSERT INTO Assignments (courseid, title, as_type, deadline, description)
VALUES ('2', 'Второе задание', 'Домашнее задание', '2025-10-12', 'Начальные
навыки - отработка');
INSERT INTO Assignments (courseid, title, as_type, deadline, description)
VALUES ('2', 'Проверка', 'Контрольная работа', '2025-10-28', 'Проверка
знаний');
INSERT INTO Assignments (courseid, title, as_type, deadline, description)
VALUES ('3', 'Первое задание', 'Домашнее задание', '2025-09-28', 'Начальные
навыки');
INSERT INTO Assignments (courseid, title, as_type, deadline, description)
VALUES ('3', 'Второе задание', 'Домашнее задание', '2025-10-12', 'Начальные
навыки - отработка');
INSERT INTO Assignments (courseid, title, as_type, deadline, description)
VALUES ('3', 'Проверка', 'Контрольная работа', '2025-10-28', 'Проверка
знаний');
INSERT INTO Assignments (courseid, title, as_type, deadline, description)
VALUES ('4', 'Первое задание', 'Домашнее задание', '2025-09-28', 'Начальные
навыки');
INSERT INTO Assignments (courseid, title, as_type, deadline, description)
VALUES ('4', 'Второе задание', 'Домашнее задание', '2025-10-12', 'Начальные
навыки - отработка');
INSERT INTO Assignments (courseid, title, as_type, deadline, description)
VALUES ('4', 'Проверка', 'Контрольная работа', '2025-10-28', 'Проверка
знаний');
INSERT INTO Assignments (courseid, title, as_type, deadline, description)
VALUES ('5', 'Первое задание', 'Домашнее задание', '2025-09-28', 'Начальные
навыки');
INSERT INTO Assignments (courseid, title, as_type, deadline, description)
VALUES ('5', 'Второе задание', 'Домашнее задание', '2025-10-12', 'Начальные
навыки - отработка');
INSERT INTO Assignments (courseid, title, as_type, deadline, description)
VALUES ('5', 'Проверка', 'Контрольная работа', '2025-10-28', 'Проверка
знаний');
INSERT INTO Assignments (courseid, title, as_type, deadline, description)
VALUES ('6', 'Первое задание', 'Домашнее задание', '2025-09-28', 'Начальные
навыки');
INSERT INTO Assignments (courseid, title, as_type, deadline, description)
VALUES ('6', 'Второе задание', 'Домашнее задание', '2025-10-12', 'Начальные
навыки - отработка');
INSERT INTO Assignments (courseid, title, as_type, deadline, description)

```

```

VALUES ('6', 'Проверка', 'Контрольная работа', '2025-10-28', 'Проверка
знаний');
INSERT INTO Assignments (courseid, title, as_type, deadline, description)
VALUES ('7', 'Первое задание', 'Домашнее задание', '2025-09-28', 'Начальные
навыки');
INSERT INTO Assignments (courseid, title, as_type, deadline, description)
VALUES ('7', 'Второе задание', 'Домашнее задание', '2025-10-12', 'Начальные
навыки - отработка');
INSERT INTO Assignments (courseid, title, as_type, deadline, description)
VALUES ('7', 'Проверка', 'Контрольная работа', '2025-10-28', 'Проверка
знаний');
#Результаты
INSERT INTO Results(Assignmentid, studentid, grade, submissiondate,
comment_fr)
VALUES ('1', '1', '99', '2025-09-23', 'Отлично!');
INSERT INTO Results(Assignmentid, studentid, grade, submissiondate,
comment_fr)
VALUES ('1', '2', '67', '2025-09-22', 'Отработай навыки!');
INSERT INTO Results(Assignmentid, studentid, grade, submissiondate,
comment_fr)
VALUES ('4', '3', '85', '2025-09-23', 'Хорошо!');
INSERT INTO Results(Assignmentid, studentid, grade, submissiondate,
comment_fr)
VALUES ('4', '4', '99', '2025-09-23', 'Отлично!');
INSERT INTO Results(Assignmentid, studentid, grade, submissiondate,
comment_fr)
VALUES ('7', '5', '34', '2025-09-22', 'Не очень хорошо, нужно стараться!');
INSERT INTO Results(Assignmentid, studentid, grade, submissiondate,
comment_fr)
VALUES ('7', '6', '78', '2025-09-23', 'Молодец!');
INSERT INTO Results(Assignmentid, studentid, grade, submissiondate,
comment_fr)
VALUES ('10', '7', '24', '2025-09-19', 'Необходима доработка!');
INSERT INTO Results(Assignmentid, studentid, grade, submissiondate,
comment_fr)
VALUES ('10', '8', '31', '2025-09-22', 'Требуется внимательность');
INSERT INTO Results(Assignmentid, studentid, grade, submissiondate,
comment_fr)
VALUES ('13', '9', '94', '2025-09-23', 'Вообще отлично');
INSERT INTO Results(Assignmentid, studentid, grade, submissiondate,
comment_fr)
VALUES ('13', '10', '12', '2025-09-21', 'Нужно больше работать!');
INSERT INTO Results(Assignmentid, studentid, grade, submissiondate,
comment_fr)
VALUES ('13', '11', '24', '2025-09-20', 'Требуется практика');
INSERT INTO Results(Assignmentid, studentid, grade, submissiondate,
comment_fr)
VALUES ('16', '12', '87', '2025-09-23', 'Молодец!');
INSERT INTO Results(Assignmentid, studentid, grade, submissiondate,
comment_fr)
VALUES ('16', '13', '44', '2025-09-21', 'Вполне неплохо! Задание было
сложным');
#Менторы
INSERT INTO Mentors (FIO, SubjectArea, Email, AppointmentDate)
VALUES
('Кузнецов Сергей Иванович', 'Программирование',
'mentor.sergey.kuznetsov@example.com', CURRENT_DATE),
('Смирнова Анна Петровна', 'Школьная математика',
'mentor.anna.smirnova@example.com', CURRENT_DATE),
('Павлов Дмитрий Николаевич', 'Высшая математика',
'mentor.dmitry.pavlov@example.com', CURRENT_DATE),

```

```

('Волкова Екатерина Андреевна', 'Информатика и ИКТ',
'mentor.ekaterina.volkova@example.com', CURRENT_DATE),
('Соколова Мария Сергеевна', 'Английский язык',
'mentor.maria.sokolova@example.com', CURRENT_DATE),
('Ильин Андрей Владимирович', 'Школьная физика',
'mentor.andrey.ilyin@example.com', CURRENT_DATE),
('Григорьева Наталья Алексеевна', 'Программирование',
'mentor.natalia.grigorieva@example.com', CURRENT_DATE),
('Захаров Михаил Дмитриевич', 'Информатика и ИКТ',
'mentor.mikhail.zakharov@example.com', CURRENT_DATE),
('Киселева Ольга Владимировна', 'Школьная математика',
'mentor.olga.kiseleva@example.com', CURRENT_DATE),
('Тихонов Виктор Сергеевич', 'Высшая математика',
'mentor.viktor.tikhonov@example.com', CURRENT_DATE);
#Расписание
INSERT INTO Schedule (GroupID, TeacherID, CourseID, Date, Time, LessonType)
VALUES
('8', '1', '1', '2025-10-01', '17:00', 'Лекция'),
('8', '1', '1', '2025-10-03', '19:00', 'Практика'),
('9', '2', '2', '2025-10-02', '17:00', 'Лекция'),
('9', '2', '2', '2025-10-04', '19:00', 'Практика'),
('10', '3', '3', '2025-10-01', '17:00', 'Лекция'),
('11', '3', '3', '2025-10-05', '19:00', 'Семинар'),
('12', '4', '4', '2025-10-02', '18:00', 'Лекция'),
('12', '4', '4', '2025-10-06', '20:00', 'Практика'),
('13', '5', '5', '2025-10-01', '18:00', 'Лекция'),
('13', '5', '5', '2025-10-07', '19:00', 'Практика'),
('14', '6', '6', '2025-10-03', '17:00', 'Лекция'),
('14', '6', '6', '2025-10-08', '19:00', 'Семинар');
#Связь учеников и групп
INSERT INTO StudentGroups (StudentID, GroupID) VALUES (1, 8), (2, 8);
INSERT INTO StudentGroups (StudentID, GroupID) VALUES (3, 9), (4, 9);
INSERT INTO StudentGroups (StudentID, GroupID) VALUES (5, 10), (6, 10);
INSERT INTO StudentGroups (StudentID, GroupID) VALUES (7, 11), (8, 11);
INSERT INTO StudentGroups (StudentID, GroupID) VALUES (9, 12), (10, 12);
INSERT INTO StudentGroups (StudentID, GroupID) VALUES (11, 13), (12, 13);
INSERT INTO StudentGroups (StudentID, GroupID) VALUES (13, 14);
#Связь менторов и результатов
INSERT INTO MentorResults (MentorID, ResultID) VALUES
(1, 2), (2, 2), (3, 3),
(1, 4), (2, 5), (3, 6);
INSERT INTO MentorResults (MentorID, ResultID) VALUES
(4, 7), (5, 8), (6, 9), (4, 10);
INSERT INTO MentorResults (MentorID, ResultID) VALUES
(7, 11), (8, 12), (9, 13);
INSERT INTO MentorResults (MentorID, ResultID) VALUES
(10, 3),
(10, 13);

```

1.5 Индексирование и связь таблиц

- При создании всех таблиц были заданы первичные ключи, которые автоматически индексируются в PostgreSQL. Это гарантирует уникальность значений и ускоряет поиск по идентификаторам;

- Для связи таблиц между собой были определены внешние ключи, которые обеспечивают ссылочную целостность данных (например, CourseID в таблице Groups ссылается на Courses.CourseID);
- Поля вида tablename_id являются автоинкрементируемыми (SERIAL), то есть их не требуется заполнять вручную - значения формируются автоматически.

1.6 Создание представлений

Как было указано в лабораторной работе №1, будет создано 4 представления – для учеников, преподавателей, менторов и админов. Код создания представлений представлен ниже в листинге.

Листинг 4 – Создание представлений

```
#Ученики
CREATE OR REPLACE VIEW student_view AS
SELECT
    s.StudentID,
    s.FIO AS student_name,
    g.Title AS group_name,
    c.Title AS course_name,
    sch.Date,
    sch.Time,
    sch.LessonType,
    a.Title AS assignment_title,
    a.Deadline,
    r.Grade,
    r.SubmissionDate,
    r.Comment_fr
FROM Students s
JOIN StudentGroups sg ON s.StudentID = sg.StudentID
JOIN Groups g ON sg.GroupID = g.GroupID
JOIN Courses c ON g.CourseID = c.CourseID
LEFT JOIN Schedule sch ON g.GroupID = sch.GroupID
LEFT JOIN Assignments a ON c.CourseID = a.CourseID
LEFT JOIN Results r ON a.AssignmentID = r.AssignmentID AND s.StudentID =
r.StudentID;

#Преподаватели
CREATE OR REPLACE VIEW teacher_view AS
SELECT
    t.TeacherID,
    t.FIO AS teacher_name,
    c.CourseID,
    c.Title AS course_name,
    g.GroupID,
    g.Title AS group_name,
    s.StudentID,
    s.FIO AS student_name,
    a.AssignmentID,
    a.Title AS assignment_title,
    r.Grade,
    r.SubmissionDate
FROM Teachers t
JOIN Courses c ON t.TeacherID = c.TeacherID
LEFT JOIN Groups g ON c.CourseID = g.CourseID
```

```

LEFT JOIN StudentGroups sg ON g.GroupID = sg.GroupID
LEFT JOIN Students s ON sg.StudentID = s.StudentID
LEFT JOIN Assignments a ON c.CourseID = a.CourseID
LEFT JOIN Results r ON a.AssignmentID = r.AssignmentID AND s.StudentID =
r.StudentID;
#Менторы
CREATE OR REPLACE VIEW mentor_view AS
SELECT
    m.MentorID,
    m.FIO AS mentor_name,
    s.StudentID,
    s.FIO AS student_name,
    a.Title AS assignment_title,
    a.Deadline,
    r.Grade,
    r.SubmissionDate,
    r.Comment_fr
FROM Mentors m
JOIN MentorResults mr ON m.MentorID = mr.MentorID
JOIN Results r ON mr.ResultID = r.ResultID
JOIN Students s ON r.StudentID = s.StudentID
JOIN Assignments a ON r.AssignmentID = a.AssignmentID;
#Админы
CREATE OR REPLACE VIEW admin_view AS
SELECT
    l.LogID,
    l.UserID,
    l.UserRole,
    l.Action,
    l.ActionTime,
    CASE
        WHEN l.UserRole = 'Student' THEN (SELECT FIO FROM Students WHERE
StudentID = l.UserID)
        WHEN l.UserRole = 'Teacher' THEN (SELECT FIO FROM Teachers WHERE
TeacherID = l.UserID)
        WHEN l.UserRole = 'Mentor' THEN (SELECT FIO FROM Mentors WHERE
MentorID = l.UserID)
        ELSE 'Администратор'
    END AS user_name
FROM Logs l;

```

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была спроектирована и реализована база данных для онлайн-школы.

Были созданы основные сущности: ученики, преподаватели, курсы, группы, задания, результаты, менторы, расписание и логи.

Для обеспечения целостности данных использовались первичные и внешние ключи, а также ограничения на значения атрибутов.